

UNIDAD DE PROGRAMACIÓN 2

FUNDAMENTOS DE JAVA

Guía didáctica de la UP2 · De PSeInt a Java

```
import java.util.Scanner;

public class Hola {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        String nombre = teclado.nextLine();
        System.out.println("Hola, " + nombre); // ¡Bienvenido a Java!
    }
}
```

Rubén Pérez Ibáñez

IES Álvaro Falomir · Curso 2026/27

Módulo: Programación (0485) · 1.º DAW

Índice de contenidos

Cómo utilizar esta guía	3
1. De PSeInt a Java	3
2. IntelliJ IDEA: tu primer proyecto	4
3. Estructura de un programa Java	5
4. Comentarios, identificadores y convenciones	6
5. Variables y tipos de datos	7
6. Constantes y literales	9
7. Operadores y expresiones	9
8. Conversiones de tipo	11
9. Salida por pantalla	12
10. Entrada por teclado	13
11. La clase Math	14
12. Errores frecuentes y cómo leerlos	15
13. Ejemplos resueltos paso a paso	16
14. Banco de ejercicios	18
15. Referencia rápida	18

Cómo utilizar esta guía

Esta guía acompaña a la UP2 y tiene un objetivo muy concreto: que pases de escribir algoritmos en PSeInt a escribir **programas reales en Java** que compilan, se ejecutan y resuelven problemas sencillos de entrada, cálculo y salida.

Cada apartado sigue la misma secuencia que ya conoces de la UP1: **problema → idea → código → prueba → ejercicio**. No intentes memorizar toda la sintaxis de golpe: escribe los ejemplos en IntelliJ, ejecútalos, cambia una línea y observa qué ocurre.

Objetivo de la unidad

Al terminar deberías ser capaz de crear un proyecto en IntelliJ, declarar variables del tipo adecuado, leer datos por teclado, hacer cálculos con operadores y mostrar resultados con un formato claro.

Si quieres...	Ve al apartado
Ver la equivalencia entre PSeInt y Java	1
Crear y ejecutar tu primer proyecto	2 y 3
Saber qué tipo de dato usar	5
Entender por qué <code>7 / 2</code> da 3	7 y 8
Leer datos sin errores raros	10
Consultar la sintaxis rápidamente	15

1. De PSeInt a Java

En la UP1 aprendiste a pensar algoritmos. Java no cambia la lógica: cambia la forma de escribirla. La mayoría de lo que hacías en PSeInt tiene una traducción directa.

PSeInt	Java	Comentario
Algoritmo Saludo ... FinAlgoritmo	<pre>public class Saludo { public static void main(String[] args) { ... } }</pre>	El programa vive dentro de una clase y empieza en <code>main</code> .
Definir edad Como Entero	<code>int edad;</code>	En Java es obligatorio declarar el tipo.
Definir precio Como Real	<code>double precio;</code>	
Definir nombre Como Caracter	<code>String nombre;</code>	<code>String</code> empieza por mayúscula.
Definir ok Como Logico	<code>boolean ok;</code>	Valores <code>true</code> / <code>false</code> .
<code>edad <- 18</code>	<code>edad = 18;</code>	El <code>=</code> de Java asigna ; no compara.
Escribir "Hola"	<code>System.out.println("Hola");</code>	
Leer edad	<code>edad = teclado.nextInt();</code>	Necesita un <code>Scanner</code> .
<code>a MOD b</code>	<code>a % b</code>	Resto de la división entera.
<code>a ^ 2</code>	<code>Math.pow(a, 2)</code> o <code>a * a</code>	<code>^</code> en Java no es potencia.
Y, O, NO	<code>&&</code> , <code> </code> , <code>!</code>	

Tres diferencias que más errores provocan

Cada instrucción termina en `;`. Java distingue mayúsculas y minúsculas, así que `Edad` y `edad` son variables distintas. Y los bloques se delimitan con llaves `{ }`, no con palabras como `FinSi`.

1.1. Compilar y ejecutar

PSeInt interpretaba tu algoritmo directamente. Java hace dos pasos:

1. **Compilación.** El compilador (`javac`) revisa el código fuente (`Saludo.java`) y, si no hay errores, genera *bytecode* (`Saludo.class`).
2. **Ejecución.** La máquina virtual de Java (JVM) ejecuta ese *bytecode*. Por eso el mismo programa funciona en Windows, macOS o Linux.

IntelliJ hace ambos pasos cuando pulsas el botón ► *Run*. Si hay un error de compilación, el programa **no llega a ejecutarse**: el IDE subraya en rojo la línea y muestra un mensaje en la parte inferior.

2. IntelliJ IDEA: tu primer proyecto

Usaremos **IntelliJ IDEA Community** (gratuito) o la licencia educativa de Ultimate.

2.1. Crear el proyecto

1. *File* → *New* → *Project...*
2. Elige **Java**, ponle nombre (`up2-ejercicios`) y una ubicación que recuerdes.
3. *Build system*: **IntelliJ** (más adelante usaremos Maven).

4. *JDK*: elige uno instalado (21 o superior). Si no aparece ninguno, *Download JDK...* y selecciona una distribución (por ejemplo, Eclipse Temurin).
5. Desmarca *Add sample code* si quieres empezar desde cero.

2.2. Crear una clase y ejecutarla

1. Clic derecho sobre la carpeta `src` → *New* → *Java Class* → escribe `Saludo`.
2. Escribe `psvm` y pulsa Tab: IntelliJ genera el método `main`.
3. Dentro de `main`, escribe `sout` y pulsa Tab: genera `System.out.println();`.
4. Ejecuta con el triángulo verde que aparece junto a `main` (o Mayús + F10).

Atajo / plantilla	Qué hace
<code>psvm</code> + Tab	Crea <code>public static void main(String[] args)</code>
<code>sout</code> + Tab	Crea <code>System.out.println();</code>
<code>souf</code> + Tab	Crea <code>System.out.printf("");</code>
Ctrl + Alt + L (⌘⌘L en macOS)	Reformatea el código (sangría y espacios)
Alt + Intro (⌘↵)	Muestra soluciones rápidas para el error subrayado
Ctrl + / (⌘/)	Comenta o descomenta la línea

Consejo

Pulsa **Ctrl + Alt + L** antes de entregar cualquier ejercicio. Un código bien sangrado se lee mucho mejor y facilita encontrar llaves mal cerradas.

3. Estructura de un programa Java

3.1. Forma clásica

```
public class Saludo {
    public static void main(String[] args) {
        System.out.println("Hola, mundo");
    }
}
```

Parte	Significado
<code>public class Saludo</code>	Define una clase pública llamada <code>Saludo</code> . El archivo debe llamarse <code>Saludo.java</code> .
<code>{ ... }</code>	Las llaves delimitan el contenido de la clase y del método.
<code>public static void main(String[] args)</code>	Punto de entrada: la JVM empieza a ejecutar aquí.
<code>System.out.println(...)</code>	Escribe un texto en la consola y salta de línea.
<code>;</code>	Fin de la instrucción.

De momento, copia la cabecera de `main` tal cual. En las UP4 y UP5 entenderás qué significan `static`, `void` y `String[] args`.

3.2. Forma compacta (Java 25 y posteriores)

Desde Java 25 se puede escribir un programa sencillo sin declarar la clase y con un `main` más corto. Además, la clase `IO` simplifica la entrada y salida:

```
void main() {
    String nombre = IO.readLine("¿Cómo te llamas? ");
    IO.println("Hola, " + nombre);
}
```

Es útil para los primeros ejercicios, y la web de la UP2 la acepta. Aun así, en clase usaremos sobre todo la forma clásica, porque es la que encontrarás en la mayoría de proyectos y la que necesitaremos cuando trabajemos con objetos.

Cuidado

La forma compacta solo funciona con un JDK 25 o superior. Si IntelliJ marca error en `void main()`, revisa en *File* → *Project Structure* → *Project* que el SDK y el *Language level* sean 25.

4. Comentarios, identificadores y convenciones

4.1. Comentarios

```
// Comentario de una línea

/* Comentario
   de varias líneas */

/**
 * Comentario de documentación (Javadoc).
 * Se usa para documentar clases y métodos.
 */
```

Comenta el **porqué** y no lo evidente. `// suma 1 a i` no aporta nada; `// el primer día del mes es el 1, no el 0` sí.

4.2. Identificadores

Un identificador es el nombre que das a una variable, constante, método o clase.

- Puede contener letras, dígitos, `_` y `$`, pero **no puede empezar por un dígito**.
- No puede coincidir con una palabra reservada (`int`, `class`, `if`, `public` ...).
- Distingue mayúsculas y minúsculas.

Elemento	Convención	Ejemplos
Variables y métodos	<i>lowerCamelCase</i>	<code>precioFinal</code> , <code>numeroAlumnos</code> , <code>calcularMedia</code>
Clases	<i>UpperCamelCase</i>	<code>Saludo</code> , <code>CalculadoraIva</code>
Constantes	MAYÚSCULAS con <code>_</code>	<code>IVA</code> , <code>PRECIO_MAXIMO</code>

Nombres significativos

`double p1, p2;` obliga a adivinar. `double precioSinIva, precioConIva;` se explica solo. Evita también tildes y la ñ en los identificadores: Java las admite, pero pueden dar problemas al cambiar de sistema operativo.

5. Variables y tipos de datos

Una **variable** es un espacio de memoria con nombre, tipo y valor. En Java se declara indicando primero el tipo:

```
int edad;           // declaración
edad = 18;          // asignación
double altura = 1.75; // declaración con inicialización
```

5.1. Tipos primitivos

Tipo	Ocupa	Rango / valores	Uso habitual
<code>byte</code>	1 byte	-128 a 127	Datos binarios
<code>short</code>	2 bytes	-32 768 a 32 767	Poco usado
<code>int</code>	4 bytes	$\approx \pm 2\,100$ millones	Enteros por defecto
<code>long</code>	8 bytes	$\approx \pm 9 \cdot 10^{18}$	Enteros muy grandes (<code>3_000_000_000L</code>)
<code>float</code>	4 bytes	~7 cifras significativas	Poco usado (<code>2.5f</code>)
<code>double</code>	8 bytes	~15 cifras significativas	Decimales por defecto
<code>char</code>	2 bytes	Un carácter Unicode	<code>'A'</code> , <code>'7'</code> , <code>'ñ'</code>
<code>boolean</code>	—	<code>true</code> / <code>false</code>	Condiciones

5.2. El tipo `String`

`String` no es un tipo primitivo, sino una **clase** que representa cadenas de texto. Se escribe con comillas dobles; un `char` va entre comillas simples.

```
String ciclo = "DAW";
char grupo = 'A';
String mensaje = "Estudio " + ciclo + " en el grupo " + grupo;
```

El operador `+` con un `String` **concatena**: une textos y convierte a texto los números que encuentre.

5.3. ¿Qué tipo elijo?

Dato	Tipo recomendado
Edad, unidades, número de alumnos	<code>int</code>
Precio, nota, temperatura, altura	<code>double</code>
Nombre, DNI, teléfono, código postal	<code>String</code> (no se opera con ellos)
Respuesta sí/no, aprobado/suspendido	<code>boolean</code>
Una letra suelta (opción de menú)	<code>char</code>

Error clásico

Un teléfono o un código postal **no** son números aunque solo tengan dígitos: no se suman ni se multiplican, y un `int` perdería el cero inicial de `08001`. Guárdalos como `String`.

5.4. Inferencia con `var`

Dentro de un método puedes escribir `var` y Java deduce el tipo por el valor inicial: `var total = 0.0;` crea un `double`. Úsalo solo cuando el tipo sea evidente; al principio es mejor escribir el tipo

explícitamente.

6. Constantes y literales

6.1. Constantes con `final`

Una constante es una variable cuyo valor no puede cambiar después de asignarlo:

```
final double IVA = 0.21;
final int MAX_INTENTOS = 3;
```

Si intentas `IVA = 0.10;`, el compilador da error. Las constantes evitan los **números mágicos**: un `0.21` repartido por el código obliga a buscarlo en todas partes si cambia el impuesto.

6.2. Literales

Literal	Tipo	Ejemplo
Entero	<code>int</code>	<code>42</code> , <code>-7</code> , <code>1_000_000</code>
Entero largo	<code>long</code>	<code>9_000_000_000L</code>
Decimal	<code>double</code>	<code>3.14</code> , <code>2e3</code> (2000.0)
Decimal simple	<code>float</code>	<code>3.14f</code>
Carácter	<code>char</code>	<code>'a'</code> , <code>'\n'</code>
Texto	<code>String</code>	<code>"Hola"</code>
Lógico	<code>boolean</code>	<code>true</code> , <code>false</code>

En el código, el separador decimal **siempre es el punto**: `3.5`, nunca `3,5`.

7. Operadores y expresiones

7.1. Aritméticos

Operador	Significado	Ejemplo	Resultado
<code>+</code>	Suma (o concatenación)	<code>7 + 2</code>	<code>9</code>
<code>-</code>	Resta	<code>7 - 2</code>	<code>5</code>
<code>*</code>	Multiplicación	<code>7 * 2</code>	<code>14</code>
<code>/</code>	División	<code>7 / 2</code>	<code>3</code> (¡entera!)
<code>/</code>	División	<code>7.0 / 2</code>	<code>3.5</code>
<code>%</code>	Resto	<code>7 % 2</code>	<code>1</code>

7.2. La división entera

Si los **dos** operandos son enteros, `/` hace división entera y descarta los decimales. Esto es útil (separar horas y minutos, cifras de un número) y también una fuente de errores.

```
int totalSegundos = 3725;
int horas = totalSegundos / 3600;           // 1
int minutos = totalSegundos % 3600 / 60;    // 2
int segundos = totalSegundos % 60;         // 5

int suma = 7 + 8;
double mediaMal = suma / 2;                 // 7.0 (se divide como int)
double mediaBien = suma / 2.0;              // 7.5
```

Recuerda

`double media = suma / 2;` **no** arregla el problema: primero se calcula `suma / 2` con enteros (7) y después se guarda como `7.0`. Uno de los operandos debe ser decimal antes de dividir.

7.3. Asignación compuesta, incremento y decremento

Expresión	Equivale a
<code>x += 5</code>	<code>x = x + 5</code>
<code>x -= 2</code>	<code>x = x - 2</code>
<code>x *= 3</code>	<code>x = x * 3</code>
<code>x /= 2</code>	<code>x = x / 2</code>
<code>x %= 4</code>	<code>x = x % 4</code>
<code>x++</code> / <code>++x</code>	<code>x = x + 1</code>
<code>x--</code> / <code>--x</code>	<code>x = x - 1</code>

`x++` usa el valor y después incrementa; `++x` incrementa y después usa el valor. Para evitar confusiones, escríbelos en una línea propia (`contador++;`).

7.4. Relacionales y lógicos

Devuelven un `boolean` y los usarás constantemente en la UP3.

Operador	Significado	Operador	Significado
<code>==</code>	igual	<code>&&</code>	Y (ambas ciertas)
<code>!=</code>	distinto	<code> </code>	O (al menos una cierta)
<code><</code> / <code><=</code>	menor, menor o igual	<code>!</code>	NO (niega)
<code>></code> / <code>>=</code>	mayor, mayor o igual		

```
int edad = 20;
boolean mayorDeEdad = edad >= 18;           // true
boolean joven = edad >= 18 && edad < 30;      // true
```

No confundir

= asigna y == compara números. Para comparar textos **no** se usa ==, sino `texto1.equals(texto2)`; lo veremos en la UP3.

7.5. Precedencia

De mayor a menor prioridad: paréntesis → ++ -- ! → * / % → + - → relacionales → == != → && → || → asignaciones. Ante la duda, **usa paréntesis**: `(a + b) / 2` no es lo mismo que `a + b / 2`.

8. Conversiones de tipo

8.1. Conversión implícita (ampliación)

Java convierte automáticamente de un tipo «pequeño» a otro «grande» porque no se pierde información: `int → long → float → double`.

```
int unidades = 3;
double precioUnitario = 2.5;
double total = unidades * precioUnitario; // el int se convierte en double: 7.5
```

8.2. Conversión explícita (casting)

Para ir de un tipo grande a uno pequeño hay que indicarlo, porque se puede perder información:

```
double nota = 7.8;
int notaEntera = (int) nota;           // 7: se trunca, no se redondea
long redondeada = Math.round(nota);    // 8
double media = (double) 15 / 2;        // 7.5
```

8.3. De texto a número y de número a texto

```
int edad = Integer.parseInt("18");
double precio = Double.parseDouble("19.95");
String texto = String.valueOf(42);     // "42"
String otro = "" + 3.5;                // "3.5"
```

Si el texto no es un número válido ("hola", "19,95"), `parseInt` / `parseDouble` lanzan una excepción y el programa se detiene. En la UP3 aprenderás a validar la entrada antes de convertir.

9. Salida por pantalla

9.1. `print` y `println`

```
System.out.print("Sin salto ");
System.out.println("con salto al final");
System.out.println("Total: " + 25 + 10);    // Total: 2510
System.out.println("Total: " + (25 + 10));  // Total: 35
```

Observa las dos últimas líneas: el `+` se evalúa de izquierda a derecha. En cuanto aparece un `String`, el resto se concatena como texto. Los paréntesis fuerzan la suma.

9.2. Secuencias de escape

Secuencia	Resultado
<code>\n</code>	Salto de línea
<code>\t</code>	Tabulación
<code>\"</code>	Comillas dobles dentro de un texto
<code>\\</code>	Barra invertida

9.3. Salida con formato: `printf`

```
String producto = "Teclado";
int unidades = 2;
double precio = 25.5;
System.out.printf("%-10s %3d uds. %8.2f €\n", producto, unidades, unidades * precio);
```

Especificador	Significado
<code>%d</code>	Entero
<code>%f</code> , <code>%.2f</code>	Decimal, con 2 decimales
<code>%s</code>	Texto
<code>%c</code>	Carácter
<code>%n</code>	Salto de línea
<code>%8.2f</code>	Ancho mínimo 8, 2 decimales (alineado a la derecha)
<code>%-10s</code>	Ancho mínimo 10 alineado a la izquierda

Decimales con coma

`printf` usa la configuración regional del ordenador. En un equipo configurado en español verás `51,00` en lugar de `51.00`. Es correcto: es solo la forma de mostrarlo.

10. Entrada por teclado

10.1. La clase `Scanner`

```
import java.util.Scanner;

public class Datos {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Nombre: ");
        String nombre = teclado.nextLine();
        System.out.print("Edad: ");
        int edad = teclado.nextInt();
        System.out.print("Altura en metros: ");
        double altura = teclado.nextDouble();

        System.out.println(nombre + " tiene " + edad + " años y mide " + altura + " m");
    }
}
```

Método	Lee
<code>nextLine()</code>	Una línea completa de texto (con espacios)
<code>next()</code>	Una palabra (hasta el primer espacio)
<code>nextInt()</code>	Un <code>int</code>
<code>nextDouble()</code>	Un <code>double</code>
<code>nextBoolean()</code>	<code>true</code> / <code>false</code>

La línea `import java.util.Scanner;` va al principio del archivo, antes de la clase. IntelliJ la añade sola si pulsas **Alt + Intro** sobre `Scanner`.

10.2. Trampa 1: `nextInt()` seguido de `nextLine()`

`nextInt()` lee el número, pero deja en el teclado el **Intro** que pulsaste. Si después llamas a `nextLine()`, este lee esa línea vacía y parece que «se salta» la pregunta.

```
System.out.print("Edad: ");
int edad = teclado.nextInt();
teclado.nextLine();           // consume el Intro pendiente
System.out.print("Nombre completo: ");
String nombre = teclado.nextLine();
```

Otra opción es leer siempre líneas y convertir: `int edad = Integer.parseInt(teclado.nextLine());`.

10.3. Trampa 2: punto o coma decimal

`nextDouble()` también depende de la configuración regional. En un ordenador en español espera **coma** (`1,75`) y falla con `1.75` (`InputMismatchException`). Si quieres que siempre acepte el punto:

```
Scanner teclado = new Scanner(System.in).useLocale(java.util.Locale.US);
```

10.4. Entrada con IO (Java 25)

En la forma compacta, `IO.readLine("Mensaje: ")` muestra el mensaje y devuelve la línea escrita como `String`. Para números, conviértela:

```
int edad = Integer.parseInt(IO.readLine("Edad: "));
```

11. La clase Math

`Math` agrupa funciones matemáticas listas para usar. No hace falta `import`.

Expresión	Resultado	Para qué
<code>Math.PI</code>	3.141592...	Constante π
<code>Math.pow(2, 10)</code>	1024.0	Potencia
<code>Math.sqrt(16)</code>	4.0	Raíz cuadrada
<code>Math.abs(-7)</code>	7	Valor absoluto
<code>Math.max(3, 9) / Math.min(3, 9)</code>	9 / 3	Mayor y menor
<code>Math.round(2.5)</code>	3	Redondeo al entero más cercano
<code>Math.floor(2.9) / Math.ceil(2.1)</code>	2.0 / 3.0	Redondeo hacia abajo / arriba
<code>Math.random()</code>	[0, 1)	Número aleatorio decimal

Para redondear a dos decimales: `Math.round(valor * 100) / 100.0`.

Ejemplo: área y perímetro de un círculo

Entrada: el radio. **Proceso:** $\text{área} = \pi \cdot r^2$, $\text{perímetro} = 2 \cdot \pi \cdot r$. **Salida:** ambos valores con 2 decimales.

```
import java.util.Scanner;

public class Circulo {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.print("Radio: ");
        double radio = teclado.nextDouble();

        double area = Math.PI * Math.pow(radio, 2);
        double perimetro = 2 * Math.PI * radio;

        System.out.printf("Área: %.2f%n", area);
        System.out.printf("Perímetro: %.2f%n", perimetro);
    }
}
```

Prueba: con radio 2 → área 12,57 y perímetro 12,57.

12. Errores frecuentes y cómo leerlos

Hay tres tipos de errores, y cada uno se detecta en un momento distinto.

Tipo	Cuándo aparece	Ejemplo	Cómo se resuelve
De compilación	Antes de ejecutar; IntelliJ lo subraya en rojo	Falta <code>;</code> , variable no declarada, tipos incompatibles	Lee el mensaje y la línea indicada.
De ejecución	Al ejecutar; el programa se detiene con una excepción	Escribir <code>hola</code> cuando se espera un número	Lee la primera línea de la traza (<code>Exception in thread "main" ...</code>).
Lógico	El programa termina, pero el resultado es incorrecto	Media con división entera	Prueba con datos cuyo resultado conoces; haz una traza.

Mensaje	Causa probable
<code>';' expected</code>	Falta el punto y coma al final de la línea anterior.
<code>cannot find symbol</code>	Nombre mal escrito (mayúsculas) o variable no declarada.
<code>incompatible types: possible lossy conversion from double to int</code>	Guardas un decimal en un <code>int</code> sin <i>casting</i> .
<code>variable x might not have been initialized</code>	Usas una variable antes de darle valor.
<code>class Saludo is public, should be declared in a file named Saludo.java</code>	El nombre de la clase y el del archivo no coinciden.
<code>InputMismatchException</code>	El dato tecleado no es del tipo esperado (o punto/coma decimal).
<code>NumberFormatException</code>	<code>parseInt</code> / <code>parseDouble</code> con un texto que no es un número.

13. Ejemplos resueltos paso a paso

13.1. Precio con IVA

Problema: pedir el precio de un producto y mostrar el IVA (21 %) y el total.

Idea: constante para el IVA; `iva = precio × IVA`; `total = precio + iva`.

```
import java.util.Scanner;

public class PrecioConIva {
    public static void main(String[] args) {
        final double IVA = 0.21;
        Scanner teclado = new Scanner(System.in);

        System.out.print("Precio sin IVA: ");
        double precio = teclado.nextDouble();

        double importeIva = precio * IVA;
        double total = precio + importeIva;

        System.out.printf("IVA (21 %%): %.2f €\n", importeIva);
        System.out.printf("Total: %.2f €\n", total);
    }
}
```

Prueba: 100 → IVA 21,00 € y total 121,00 €. Observa `%%`: así se escribe un símbolo `%` dentro de `printf`.

13.2. Intercambio de dos variables

Problema: intercambiar los valores de `a` y `b`.

Idea: si hacemos `a = b` perdemos el valor de `a`. Necesitamos una variable auxiliar.

```
public class Intercambio {
    public static void main(String[] args) {
        int a = 8;
        int b = 3;
        int aux = a;    // guardamos a
        a = b;           // a vale 3
        b = aux;         // b vale 8
        System.out.println("a = " + a + ", b = " + b);
    }
}
```

Instrucción	a	b	aux
inicio	8	3	—
<code>aux = a</code>	8	3	8
<code>a = b</code>	3	3	8
<code>b = aux</code>	3	8	8

13.3. Segundos a horas, minutos y segundos

```
import java.util.Scanner;

public class Tiempo {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.print("Segundos: ");
        int total = teclado.nextInt();

        int horas = total / 3600;
        int resto = total % 3600;
        int minutos = resto / 60;
        int segundos = resto % 60;

        System.out.println(horas + " h " + minutos + " min " + segundos + " s");
    }
}
```

Prueba: 3725 → 1 h 2 min 5 s .

14. Banco de ejercicios

Los ejercicios de esta unidad están en la web de la UP2, con pistas en la plantilla, validación por criterios y justificante:

- **Práctica 1 · Sentencias simples (11):** nombre y edad, suma, cinco operaciones, área del rectángulo, círculo, intercambio, informe de ventas, IVA fijo, IVA variable, euros a pesetas y las dos piscinas.
- **Ampliación (12 retos):** ticket con propina, segundos a h/min/s, cajero, notas ponderadas, temperaturas, cifras de un número, viaje, carrera, rotación de tres variables, factura de la luz y distancia entre puntos, entre otros.

Método de trabajo

Resuelve en IntelliJ, ejecuta con varios datos (incluido algún caso límite), pega el código en la web, valida y corrige. La web no compila tu programa: la prueba de verdad es ejecutarlo.

15. Referencia rápida

```
import java.util.Scanner;                                // al principio del archivo

public class Plantilla {
    public static void main(String[] args) {
        final double CONSTANTE = 0.21;                  // constante
        Scanner teclado = new Scanner(System.in);

        int entero = teclado.nextInt();                  // leer entero
        double decimal = teclado.nextDouble();           // leer decimal
        teclado.nextLine();                              // limpiar el Intro
        String texto = teclado.nextLine();               // leer línea

        double resultado = entero * decimal * CONSTANTE;
        System.out.println("Texto: " + texto);
        System.out.printf("Resultado: %.2f%n", resultado);
    }
}
```

Necesito...	Escribo...
Resto de una división	<code>a % b</code>
División con decimales entre enteros	<code>(double) a / b</code> o <code>a / 2.0</code>
Potencia	<code>Math.pow(base, exp)</code>
Raíz cuadrada	<code>Math.sqrt(x)</code>
Redondear a 2 decimales al mostrar	<code>printf("%.2f", x)</code>
Convertir texto a número	<code>Integer.parseInt(s)</code> , <code>Double.parseDouble(s)</code>
Truncar un decimal	<code>(int) x</code>